Technical report: a sequential matching policy under uncertainty

Team: Danpotatoes: Dan George (roll no. 25f3001550) and Pranav Mesta (roll no. 24f1002540).
Repository: https://github.com/24f1002540/sequential-matching-policy **Release:** participant kit 1.0.0 (synthetic data only). **Submitted version:** see the commit SHA in the Final submission issue.

1. Summary

We submit a deliberately simple, deterministic policy: reciprocal feasibility from the supplied eligibility rules, targeted hard-constraint clarification, supplied similarity-greedy allocation, and an independent output guard. It needs no learned asset, no network and no random numbers. The choice follows from our Round 1 research and from additional experiments in this report:

- **The score is supply-bound.** An episode contains about 100 pairs that are ever feasible and about 80% are introduced, so allocation and ranking rules have little to choose between (Section 5.2).
- **Clarification is essential** (no-clarification: coverage 0.14 against about 0.35).
- **No candidate we tested was distinguishable from the supplied baseline on the primary score**, including learned edge values, global matching, online adaptation and scarcity-weighted matching (Section 5).
- **Validity is the decisive risk:** one invalid assessed episode makes an entry ineligible, so the policy is built around a guard that can only return valid actions.

2. Problem and assumptions

The policy runs for 60 decision days on a changing population of 200 synthetic members. Each day it makes two calls: an *ask* call (clarification requests within 12 units; the hard-constraint bundle costs 3, a soft field 1) and a *match* call (disjoint feasible pairs, possibly none). It observes only the request JSON: arrived members, field values with statuses (observed, not asked, declined) and observation days, past introductions and delayed feedback, and its own `memory`.

The primary score is mutual second-meeting intentions (MSMI) per 100 arrived members: both members accept the introduction, a date happens within 30 days of assignment, and both return Yes to a second meeting within 3 days of the date. The score is the equal-weight mean over six scenario families. Ties are broken by coverage, then mutual acceptances per 100, then lower clarification cost, then lower inference time.

Assumptions we rely on, each checked against the public kit documentation: (i) `null` means unknown, never zero or rejection; (ii) a declined field stays unavailable, so asking again is wasted budget and matching on it is prohibited; (iii) hard constraints are reciprocal and must be satisfied in both directions; (iv) the hidden outcome process, response propensities and seeds are never observable, and the policy never tries to reconstruct them.

3. Method

1. **Reciprocal feasibility.** The supplied `eligibility(a, b)` is applied in both directions. A known hard failure excludes the pair; a missing hard field makes it `needs_clarification`; only `feasible` pairs can be introduced. Soft fields never override hard rules.
2. **Clarification.** For every pair of available members that has not met and still needs clarification, each member who must answer receives credit 1/(number of members in that pair who must answer). Pairs involving a member who declined a hard field are skipped, since that field can never be learned. Members are ranked by total credit (ties by identifier) and asked the 3-unit bundle until fewer than 3 units remain. This is a simple value-of-information rule: ask where an answer can unlock many pairs, nowhere else.
3. **Allocation.** The supplied greedy matcher scores each feasible pair by the number of soft fields on which both known values agree and takes disjoint pairs in descending order. The default is to fill: an introduction costs nothing directly, and our supply analysis shows members are rarely idle with an option, so waiting rules have little room to help.
4. **Guard.** An independent check re-validates every ask (member exists and is available, valid field, no duplicate, within budget) and every pair (both available, feasible, not repeated, no reuse in the batch) and returns the empty action on any exception.
5. **Memory.** None is needed: every call recomputes decisions from the full observable state, which keeps the policy restart-safe and far below the 1 MiB limit.

We did not use a learned probability model in the submission. Our held-out checks (Section 5.2) showed only a weak first-stage signal (AUC 0.575) and no usable second-meeting signal (AUC 0.52), and models gave no detectable gain in the full simulator.

4. Experimental design

- **Simulator and seeds.** All experiments use the public simulator (episodes of 200 members, 60 decision days, 40 follow-up days) across the six scenario families. Round 1 used exploratory seeds 101 to 105 and confirmatory seeds 106 to 125. Round 2 adds fresh seeds 201 to 240 for the adoption test, and the official evaluator on seeds 101 to 105 for the final results. Training seeds (1000 to 1007) and validation seeds (2000 to 2002) were used only for the learned-model checks and are separate from all evaluation seeds.
- **Paired comparisons.** Every policy runs on identical seeds and variants. Intervals are 95% percentile-bootstrap intervals that resample *seeds* (each seed contributes its six family results together), which is more conservative than resampling episodes.

- **Pre-specification.** The first four contrasts (K1 to K4) were written down before any confirmatory episode ran; later ones are dated addenda (`PRESPEC.txt`). The Round 2 adoption rule for the candidate upgrade was written before any seed-201-to-240 episode ran (`PRESPEC_R2.txt`).
- **Faithfulness.** In-process runs of the supplied baselines reproduced the supplied evaluator exactly on the episodes we compared, and our policy was run through the official JSON subprocess protocol for every result in Section 6.

5. Evidence behind the design

5.1 The score is supply-bound

On 24 episodes (seeds 106 to 109, six families) we tracked which pairs were ever feasible among available members and which were introduced. On average 100.2 pairs are ever feasible per episode, 80.3 (80%) are introduced, and an offline supply bound (every non-declined constraint known) is 113.3, so clarification already reaches 88% of what could be feasible. On a typical day only about 2.7 feasible pairs exist among 200 members, and members sit idle with an available option on 0.49% of member-days. On average 32% of members (range 27% to 47%) decline at least one hard field and can never be introduced. The funnel is also thin: of 8,822 introductions by the supplied baseline over 120 confirmatory episodes, 14.3% became mutual acceptances, 79% of those led to a date, and 11% of dates produced a mutual second-meeting Yes (113 MSMI events, about 0.9 per episode). 43% of episodes score zero.

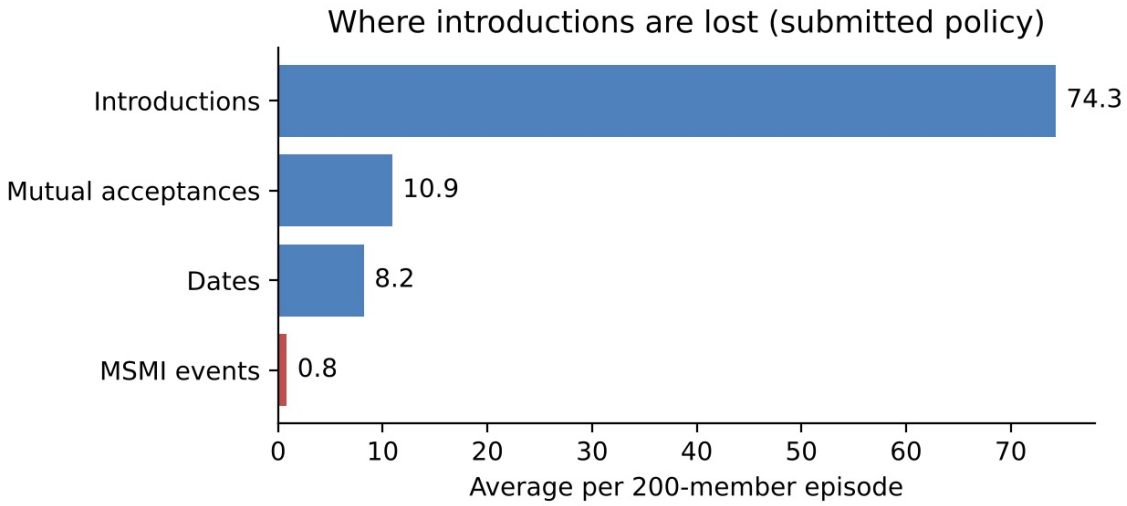


Figure 1. Average funnel per 200-member episode for the submitted policy (Round 1 confirmatory seeds): about 74 introductions yield about 11 mutual acceptances, 8 dates and 0.8 MSMI events.

5.2 Candidate comparison (Round 1 confirmatory seeds 106 to 125)

Arms: A supplied baseline asks plus greedy; C targeted hard asks plus supplied greedy (the submitted policy); Fg learned edge values plus soft asks, greedy; F the same with global maximum-weight matching; G F plus online adaptation; S targeted asks plus scarcity-weighted maximum-cardinality matching; O an offline oracle ranking (a bound, not a policy).

Table 1. Confirmatory summary (20 seeds, 120 episodes per policy).

Policy	Primary score (MSMI/100)	Coverage	Mutual acc./100	Ask cost	Introductions	Mutual acc.	Dates	MSMI events	Zero-MSMI episodes
A baseline asks + supplied greedy (supplied)	0.471	0.338	5.27	200	73.5	10.5	8.3	113	51 / 120
C targeted asks + supplied greedy (incumbent)	0.417	0.338	5.45	197	74.3	10.9	8.2	100	58 / 120
Fg learned values + soft asks, greedy	0.479	0.337	5.40	241	73.9	10.8	8.2	115	52 / 120
F learned values + soft asks, global	0.475	0.338	5.70	239	73.4	11.4	9.1	114	53 / 120
G F + online adaptation	0.446	0.339	5.55	239	74.1	11.1	8.6	107	53 / 120
S targeted asks + scarcity-weighted matching	0.342	0.340	5.53	197	75.6	11.1	8.3	82	69 / 120
O oracle ranking (offline bound, not a policy)	0.433	0.337	5.58	197	73.6	11.2	8.7	104	59 / 120

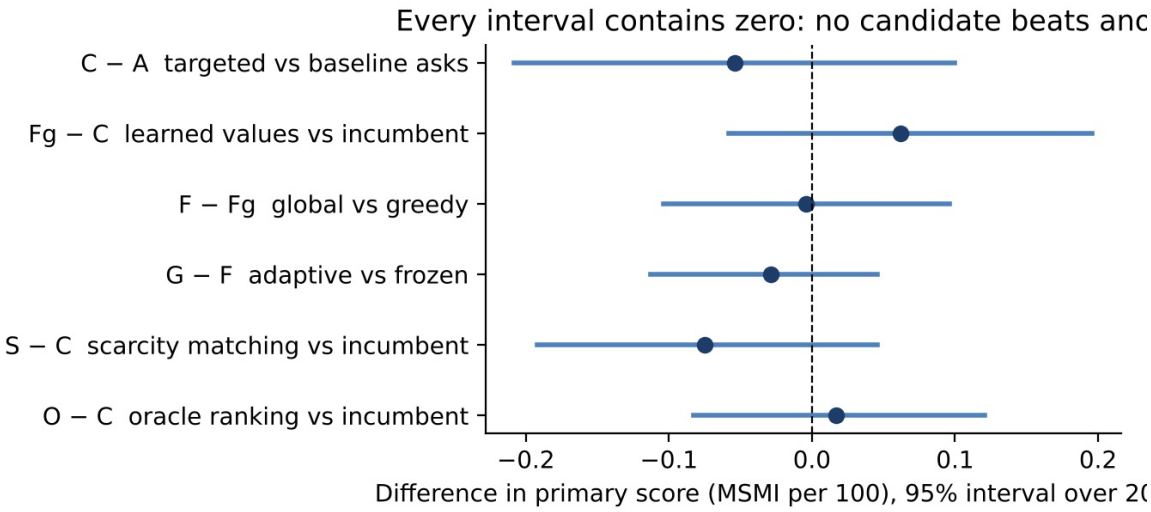


Figure 2. Paired differences in primary score (Table 2). Every interval includes zero.

Table 2. Confirmatory contrasts (first minus second; 20 seeds; MSMI events per episode, the primary score is 0.5 times this).

Contrast (first minus second)	Test	Mutual acceptances per episode	MSMI events per episode	Primary score (MSMI/100)	Seeds
C − A targeted vs baseline asks	K1, K2	+0.36 [-0.38, +1.18]	-0.11 [-0.42, +0.19]	-0.054 [-0.208, +0.100]	20
Fg − C learned values + soft asks vs incumbent	K3	-0.10 [-0.71, +0.49]	+0.12 [-0.12, +0.39]	+0.062 [-0.058, +0.196]	20
F − Fg global vs greedy, same learned values	K5	+0.60 [+0.07, +1.11]	-0.01 [-0.20, +0.19]	-0.004 [-0.104, +0.096]	20
G − F adaptive vs frozen	K6	-0.30 [-0.68, +0.07]	-0.06 [-0.23, +0.09]	-0.029 [-0.113, +0.046]	20
S − C scarcity-weighted matching vs incumbent	K10, K11	+0.17 [-0.67, +0.83]	-0.15 [-0.38, +0.09]	-0.075 [-0.192, +0.046]	20
O − C oracle ranking vs incumbent (bound)	K9	+0.26 [-0.42, +0.89]	+0.03 [-0.17, +0.24]	+0.017 [-0.083, +0.121]	20

O − C oracle ranking vs incumbent (bound)	K9	+0.26 [-0.42, +0.89]	+0.03 [-0.17, +0.24]	+0.017 [-0.083, +0.121]	20

Reading. No contrast on the primary score excludes zero. Targeted asks reduce clarification cost by 3.3 units per episode (interval [-4.2, -2.5], cheaper in 20 of 20 seeds) with no detectable change in outcomes. Global matching raises mutual acceptances (+0.60) and dates (+0.83) but not MSMI. Online adaptation slightly lowers dates (-0.48 [-0.82, -0.13]). An earlier 5-seed result that targeted asks beat the baseline on MSMI did not replicate on 20 fresh seeds (pooled over 25 seeds: -0.03 events [-0.30, +0.24]), and we withdrew it.

Clarification is essential (exploratory seeds 101 to 105): with no clarification, mutual acceptances fall by 6.40 per episode [-8.37, -4.47] and MSMI events by 0.37 [-0.53, -0.20], and coverage drops from about 0.35 to 0.14.

Learned models. Training on 2,863 randomised-among-feasible rollout introductions, the first-stage model reached AUC 0.575 on held-out rollouts (Brier 0.2315 against 0.2355 for a constant), but the second-meeting model was uninformative (AUC 0.520; Brier 0.0748 against 0.0745) with 333 training dates, and learned values gave no detectable MSMI gain (Fg minus C: +0.12 [-0.12, +0.39]). On the static data, a similarity model scored AUC 0.465 on held-out directional responses and member history AUC 0.510 [0.458, 0.561], that is, no usable signal. We therefore ship no learned asset.

5.3 Headroom

An offline expected-outcome measure (simulator true probabilities summed over the pairs a policy introduced; evaluation only, 36 episodes per policy) gives expected MSMI per episode of 0.836 (A), 0.826 (C), 0.860 (Fg), 0.859 (S) and 0.882 for the oracle O. The oracle exceeds C by only +6.8% [-0.5%, +14.2%]. Even perfect knowledge of pair quality adds about 7%, which a 20-seed evaluation cannot resolve: per-seed primary-score standard deviations are 0.28 to 0.33, giving a minimum detectable difference of about 0.22 MSMI/100 at 20 seeds per family (0.157 at 40, 0.099 at 100) against scores near 0.45.

5.4 Round 2 adoption test (hypothesis-driven ablation of the allocation rule)

Hypothesis (H-S). Scarcity-weighted matching, which prefers pairs that use members with few feasible partners, raises throughput and therefore MSMI over the supplied greedy rule, with the same clarification policy. *Pre-registered rule* (`PRESPEC_R2.txt`, seeds 201 to 240, 240 episodes per arm): adopt only if introductions rise with a lower bound above zero, MSMI events do not fall by more than 0.10 per episode (lower bound), mutual acceptances do not fall, and ask cost does not rise by more than 1 unit.

Table 3. Scarcity matching (S) against the submitted policy (C), 40 fresh seeds.

Measure (per episode)	S	C	S minus C [95% interval over seeds]
Introductions	73.59	73.35	+0.24 [-0.85, +1.33]
Mutual acceptances	10.87	10.75	+0.12 [-0.44, +0.73]
Dates	8.60	8.48	+0.11 [-0.41, +0.69]

MSMI events	0.717	0.725	-0.008 [-0.154, +0.133]
Coverage	0.332	0.331	+0.001 [0.000, +0.002]
Ask cost (units)	192.5	192.5	-0.01 [-0.10, +0.09]

Result. The introduction and MSMI conditions failed (lower bounds -0.85 and -0.154), so S was **not adopted**. The hypothesis is not supported: throughput did not rise. This matches the supply analysis. With so few feasible pairs per day, allocation has almost nothing to rearrange.

5.5 Clarification ordering: why we did not test further tweaks

The daily clarification budget (4 hard-constraint bundles) binds on some days: in a diagnostic on 6 episodes (seeds 201 and 202, standard, cold start and delayed families), 83 of 360 days had more askable members than budget, although total spending (about 195 of 720 units per episode) shows the backlog drains later. We therefore tried an alternative ordering, *pair completion* (greedy marginal credit that favours finishing pairs by choosing partners together). On 120 days of two episodes it selected a different member set on only 5 days (4%), and on the one episode we ran fully it produced identical results to the submitted rule. An effect that rarely changes any action cannot be resolved with the seeds we can afford, so we did not run a confirmatory test and kept the simpler rule. This is a diagnostic, not a pre-specified comparison (`experiments/diag_budget.py`, `experiments/diag_t.py`, `experiments/t_arm.py`).

6. Final evaluation through the official evaluator

The submitted policy and the three supplied baselines were run with the official `evaluate.py` protocol (a separate process per call, 1 MiB and 10 s limits checked), on seeds 101 to 105 and all six scenario families (30 episodes per policy). All 120 episodes were valid. Raw results: `results/final.json`, `results/baseline_greedy.json`, `results/baseline_no_asks.json`, `results/baseline_random.json`.

Read this section with Section 5. Seeds 101 to 105 are the exploratory seeds on which our targeted-asking advantage first appeared, and it did not replicate on 20 fresh seeds (Table 2). The primary-score gaps below are therefore not evidence that the submitted policy outperforms the supplied greedy baseline; they are the final-format results the submission rules ask for. The two comparisons that are robust are that **clarification matters** (no-clarification scores lowest on every measure that depends on feasible pairs) and that the submitted policy **spends fewer clarification units** than the baseline.

Table A. Primary score and ranking measures (official evaluator output, seeds 101 to 105, 30 episodes per policy).

Policy	Valid episodes	Primary score (MSMI/100)	Coverage	Mutual acc. /100	Ask cost	Mean inference s/episode	MSMI events
Submitted policy	30 / 30	0.517	0.352	5.87	195	34.0	31
Supplied greedy baseline	30 / 30	0.367	0.353	5.18	197	22.8	22
Supplied no-	30 / 30	0.183	0.137	1.98	0	24.1	11

clarification							
Supplied random-feasible	30 / 30	0.267	0.356	5.08	197	22.5	16

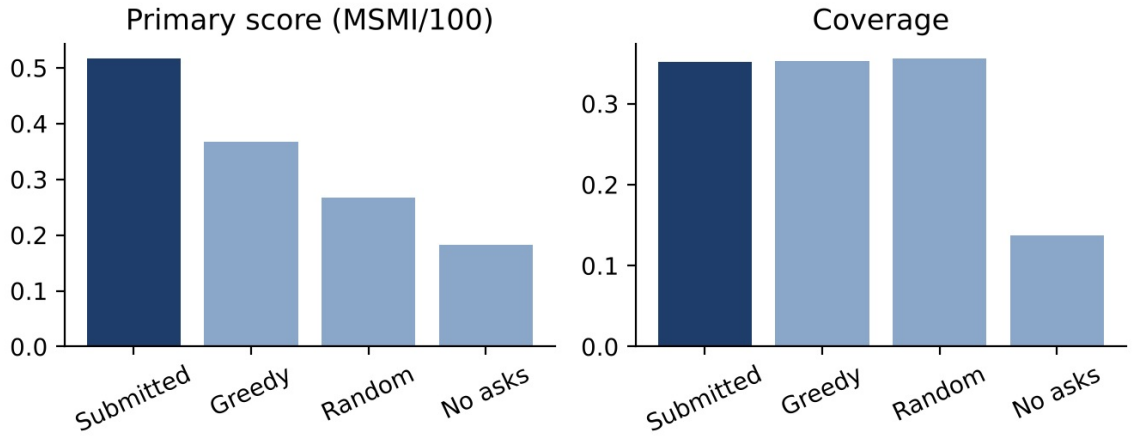


Figure 3. Table A as a chart (exploratory seeds 101 to 105; see the reading note above).

Table B. Scenario-level outcomes (means per episode).

Scenario family	Policy	Introductions	Mutual acc.	Dates	MSMI events	Coverage	Median first wait (days)	Unserved members	Missing feedback	Ask cost
standard	Submitted policy	89.6	15.4	12.2	1.40	0.400	2.9	120.0	52.4	187
standard	Supplied greedy baseline	91.8	12.4	9.6	1.00	0.403	3.8	119.4	53.6	188
standard	Supplied no-clarification	30.4	5.2	4.2	0.60	0.172	0.0	165.6	18.0	0
standard	Supplied random-feasible	92.8	11.4	9.8	0.60	0.409	4.2	118.2	50.0	188
sparse geography	Submitted policy	18.0	2.8	2.2	0.00	0.139	2.2	172.2	7.2	178
sparse geography	Supplied greedy baseline	18.0	3.6	3.0	0.20	0.139	3.4	172.2	8.6	184
sparse geography	Supplied no-clarification	5.4	0.8	0.8	0.00	0.045	1.4	191.0	2.0	0
sparse geography	Supplied random-feasible	18.0	3.4	2.8	0.20	0.139	3.5	172.2	8.6	184
cold start	Submitted policy	74.6	10.2	7.8	1.00	0.372	5.0	125.6	40.0	241
cold start	Supplied greedy baseline	79.8	10.4	9.0	0.80	0.370	6.1	126.0	42.8	246
cold start	Supplied no-clarification	15.4	2.0	1.6	0.20	0.091	0.0	181.8	8.4	0
cold start	Supplied random-feasible	78.8	11.0	10.0	0.40	0.368	6.6	126.4	45.6	246
delayed dates	Submitted policy	87.6	14.6	11.6	1.20	0.400	2.8	120.0	49.8	187
delayed dates	Supplied greedy baseline	90.0	12.2	9.8	0.60	0.402	3.7	119.6	51.6	188
delayed dates	Supplied no-clarification	30.0	5.6	4.2	0.40	0.172	0.0	165.6	17.2	0
delayed dates	Supplied random-feasible	90.6	12.8	10.2	0.60	0.408	4.2	118.4	47.2	188
shifted weights	Submitted policy	90.4	13.0	10.4	1.40	0.401	2.9	119.8	50.0	187

shifted weights	Supplied greedy baseline	93.0	11.8	9.4	0.80	0.404	3.7	119.2	52.8	188
shifted weights	Supplied no-clarification	31.2	5.0	4.2	0.40	0.173	0.0	165.4	18.4	0
shifted weights	Supplied random-feasible	93.0	11.6	9.0	0.80	0.406	4.0	118.8	48.2	188
changing conditions	Submitted policy	90.0	14.4	11.4	1.20	0.400	2.9	120.0	52.4	187
changing conditions	Supplied greedy baseline	92.4	11.8	9.0	1.00	0.403	3.8	119.4	53.6	188
changing conditions	Supplied no-clarification	30.4	5.2	4.2	0.60	0.172	0.0	165.6	18.0	0
changing conditions	Supplied random-feasible	93.0	10.8	9.2	0.60	0.409	4.2	118.2	49.8	188

Reading. The submitted policy produced no invalid episode, its slowest episode used 47.7 s of inference across 120 calls (the per-call limit is 10 s; the slowest single call in our in-process runs was 0.26 s), and its ask cost is 2 units below the supplied baseline's on these seeds (and 3.3 units on the 20 confirmatory seeds). Its per-episode inference time is higher than the supplied baselines' because it evaluates pair status for the available population in both the ask and match calls and again in the guard; this affects only the last tie-break.

6.1 Further validity, decision trace and robustness checks

- **Wider official-protocol run.** The submitted policy was run through the unmodified evaluator protocol on ten further seeds (106 to 115, six families, 60 episodes; `results/validity_a.json`, `results/validity_b.json`). All 60 episodes were valid; the slowest took 35.0 s of inference over 120 calls (mean 24.7 s). The primary score on these seeds was 0.250 (30 MSMI events), coverage 0.337 and ask cost 197.5. This is far below the 0.517 on seeds 101 to 105 and is the same size of swing that Section 7 attributes to noise; it is another reason we do not claim a primary-score advantage.
- **Fuzzing (`test_fuzz.py`).** States from simulated episodes were mutated (blank or declined fields, unavailable members, empty or tiny populations, odd budgets, duplicated introductions, shuffled order) and several malformed requests were sent. The policy returned a valid action every time, and a malformed request produced an empty valid action instead of an error.
- **Scale (`test_scale.py`).** Populations of 200 and 400 members over 20 days: slowest single call 0.35 s and 0.81 s, responses under 2 KiB. A 200-member request is about 397 KiB, so the harness's own 1 MiB request limit is reached at roughly 500 members, long before the 10 s limit would matter for this policy.
- **Decision trace (`explain.py`, `explain_episode.py`).** For every ask the trace records the member, field, cost, credit and the pairs the answer could unlock; for every introduction it records the agreeing soft fields, the similarity score and the member's remaining options. `test_explain.py` confirms the trace never changes a decision (identical actions with and without it).

7. Limitations

- **Public worlds only.** The six public scenario families approximate, but are not, the private worlds. Effects that vanish here could appear there and vice versa.
- **Noise.** About one MSMI event occurs per 200-member episode. A 20-seed-per-family evaluation resolves differences of about 0.22 MSMI per 100 against a score near 0.45, so ranking among competent policies is largely chance. We therefore do not claim that our policy outperforms the baseline on the primary score.
- **Selective labels.** Outcomes exist only for the pairs we introduce, and propensities are null in the supplied logs, so we make no inverse-propensity claims and compare complete policies on identical seeds instead.
- **Offline analyses use simulator internals** (the supply bound, the oracle ranking and the expected-outcome measure) for bounding and variance reduction only. No submitted policy reads them, and no policy parameter was fitted to them.
- **Exploratory against confirmatory.** Only K1 to K4 and the Round 2 adoption rule were specified before the confirmatory runs. Later contrasts are labelled and are less strictly confirmatory. We did not adjust for multiple comparisons.
- **Container check.** The authoring sandbox had no Docker daemon, so the container build and the offline container evaluation (`evaluate.py --image`, seed 101) were verified on GitHub Actions for the public repository: both the organisers' `verify.yml` workflow (tests on Python 3.10 and 3.12, data verification, local and container evaluation) and our `container-check.yml` workflow passed on commit 4f1c513 (an earlier commit failed because our Dockerfile copied a file that the supplied `.dockerignore` excluded; we restored the supplied Dockerfile). The container check covers one episode, not every assessed episode.
- **Synthetic only.** Nothing here estimates real-world compatibility or relationship outcomes. A real service would need an adapter, validation on authorised data, current constraint checks and human review.

8. Reproducibility

All commands are in the README, and `bash run_all.sh` runs the tests, scale check and official-protocol evaluation in one step; `make_figures.py` redraws the figures from the numbers printed here. The submitted policy is deterministic (no random numbers; ties are broken by member identifier) and has no training seeds. Experiment scripts record seeds and variants; raw per-episode results are under `results/` and `experiments/`. The pre-specification files (`PRESPEC.txt`, `PRESPEC_R2.txt`) are included with their dates.

9. Provenance and tools

Only the public kit was used; no external data or paid services. Inside the image: Python standard library only. Offline experiment scripts additionally use NumPy and NetworkX (BSD-3-Clause), run on a single CPU core. An AI assistant (Claude, Anthropic) was used to read the

repository, write and run the experiments and the policy code, and draft the Round 1 note, the code and this report. The team chose the problem track, directed the work and its scope, supplied the repository and accounts, and is responsible for the submission; the team's own review of individual results is not claimed beyond that. The team retains ownership of its work; this report grants no commercial licence.